

Welcome to

Lab5: CNNs and Transformers

Date: 18.03.2026

Who I am: Ceren Gok - PhD student working on Continual Learning and TA for MLE course 2026!

What is these Labs are about: Guided-coding exercises, Try to remember what we learned from the lectures, Sharing our opinions with our peers, aaannd competing with fun way !

The Structure of Slides: Historical Facts, Fun Facts, Revisiting Theories, Interview Questions, Live Coding Exercise for you.

Welcome to

Lab5: CNNs and Transformers

Intended Learning Outcomes:

- Identify the architectural limitations of MLPs when applied to vision
- Explain how CNNs differ from MLPs and better alternatives for vision
- Compare CNN and Vision Transformer architectures
- Select an appropriate model architecture and fine-tuning strategy given real-world constraints
- Implement pretrained CNN and ViT models

Welcome to

Lab5: CNNs and Transformers

Lets start with a Game and see how much we remember from lecture notes!



Interview Moment: A Broken Demo at 2 AM



It's 2 AM before a product demo.

Your teammate wants to build a defect **classifier** for factory parts and is arguing for an **MLP**.

They've been reading benchmarks and say:

'I'll train it on the full **256×256 production photos** and will **use MLPs** because they are **simple** and I can always go **deeper** if accuracy is bad.'

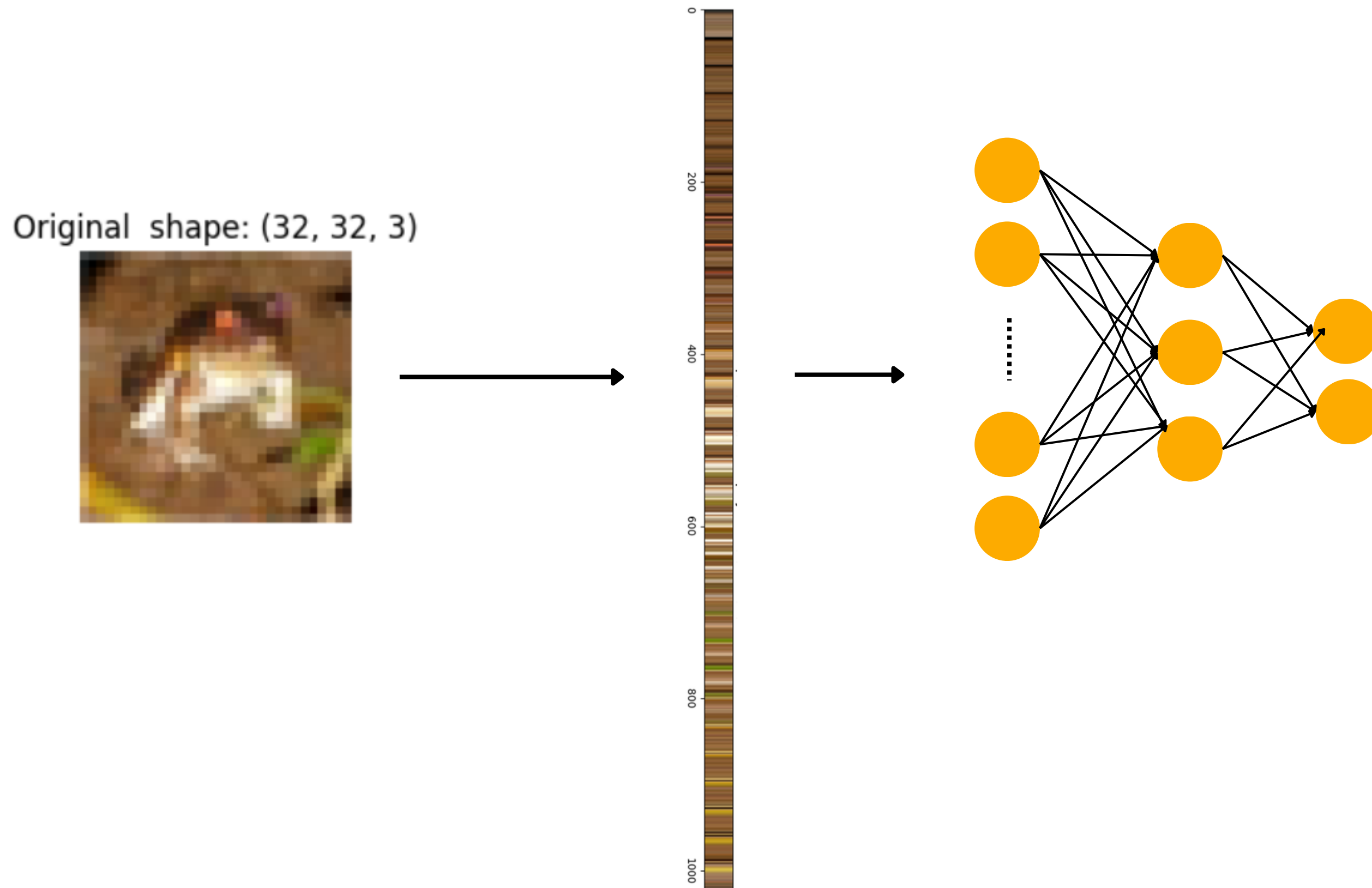
You immediately see **two reasons** why this will **fail** before the demo even starts.

What do you think these reasons are for not to use MLPs?



We could use MLP for image classification

In previous lab we did image classification on CIFAR10 dataset.



Best Model:

- ReLu
- Adam
- Kaiming init
- Cosine Annealing Scheduler
- 20 epoch
- weight decay + dropout
- **Test acc: 56%**

Is there a reason not use MLPs for vision ?

Reason1: MLPs do not learn the spatial structure in the image because of flattening.

Original shape: (32, 32, 3)



The pixels for the eye will not be neighbors anymore in the flattened image.

How CNNs overcome this ?

Solution1: CNNs learn spatial structure in the image because they operate directly on the 2D grid. Since we have the kernels that are sliding on an image we do capture relationships among neighbour pixels.

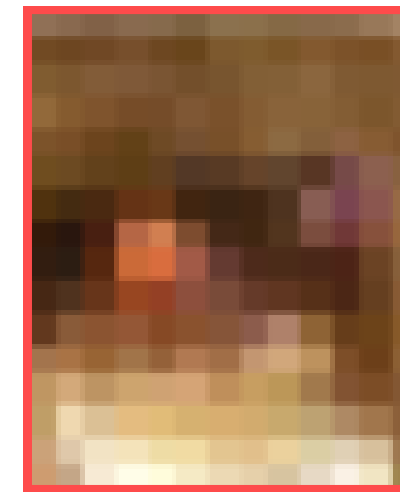
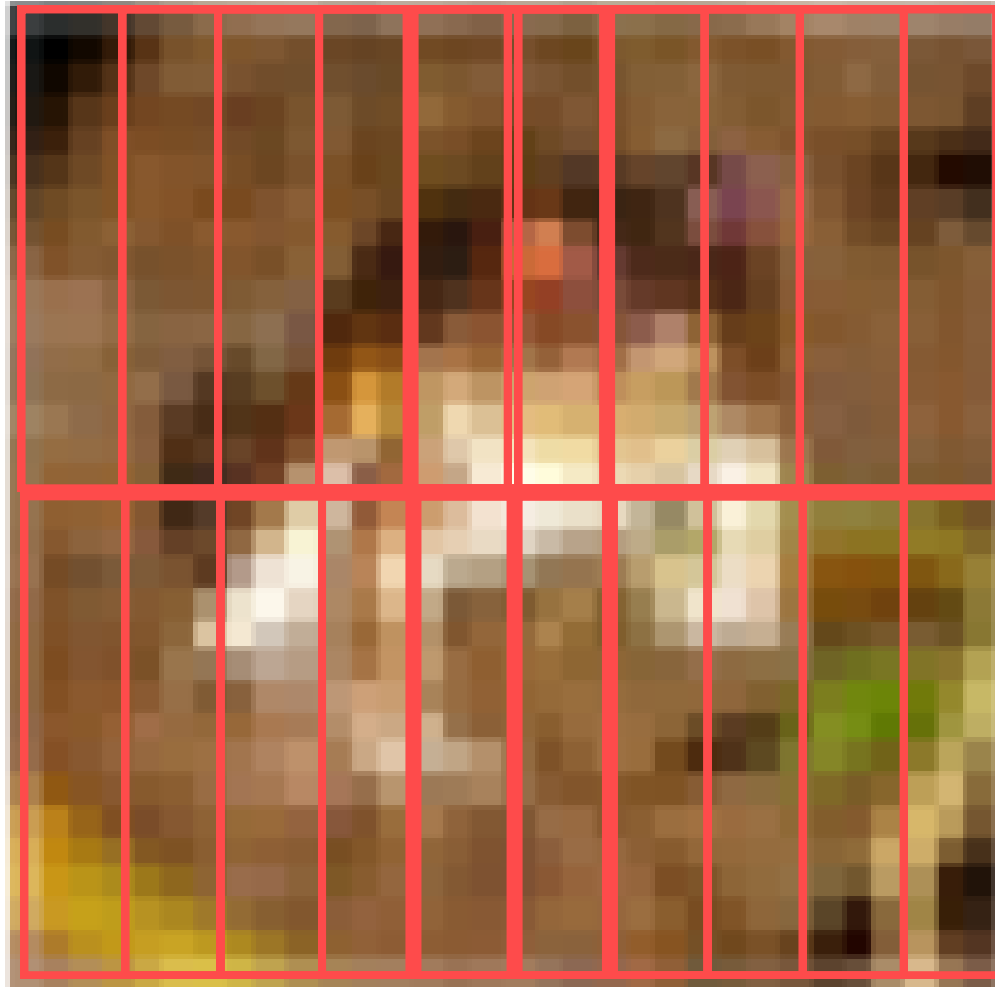
Original shape: (32, 32, 3)



How CNNs overcome this ?

Solution1: CNNs learn spatial structure in the image because they operate directly on the 2D grid. Since we have the kernels that are sliding on an image we do capture relationship among neighbour pixels.

Original shape: (32, 32, 3)



Information between eye pixels will not be lost.

Is there a reason not use MLPs for vision ?

Reason2: They explode with input size

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

4x4 input image

Is there a reason not use MLPs for vision ?

Reason2: They explode with input size

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

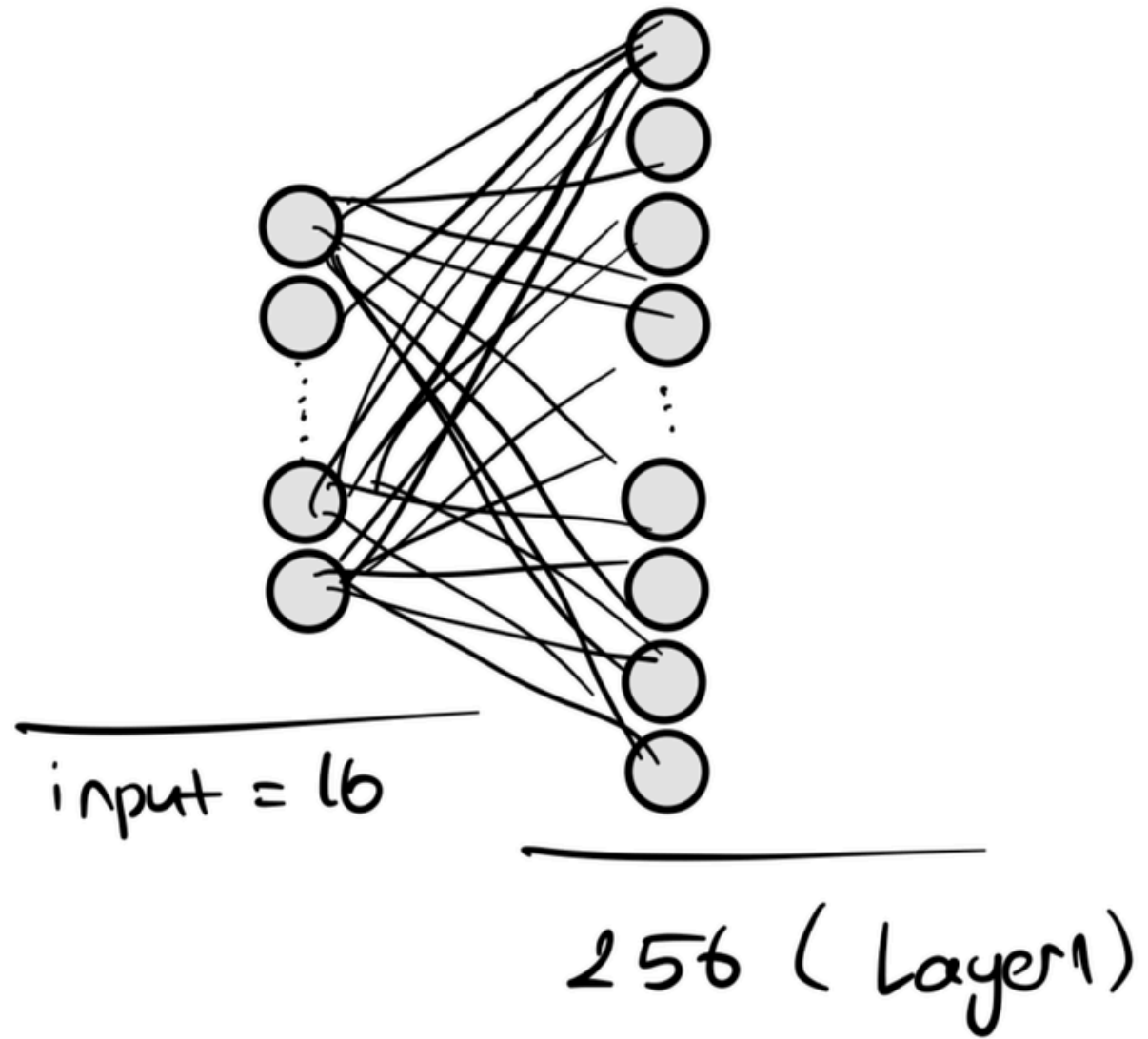
4x4 input image

$$[1 \quad 2 \quad 3 \quad \dots \quad 15 \quad 16]$$

Flattened image

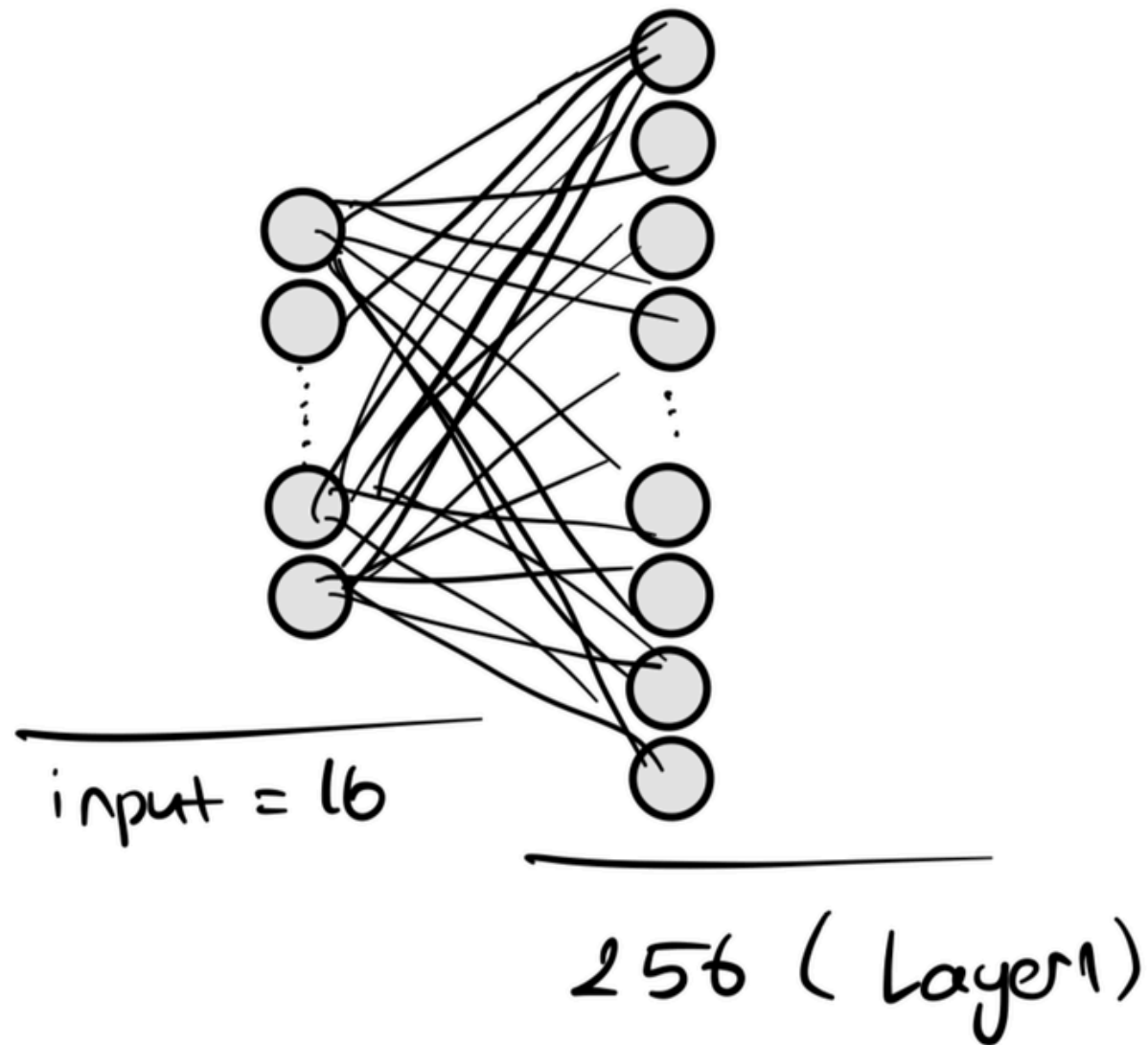
Is there a reason not use MLPs for vision ?

Reason2: They explode with input size



Is there a reason not use MLPs for vision ?

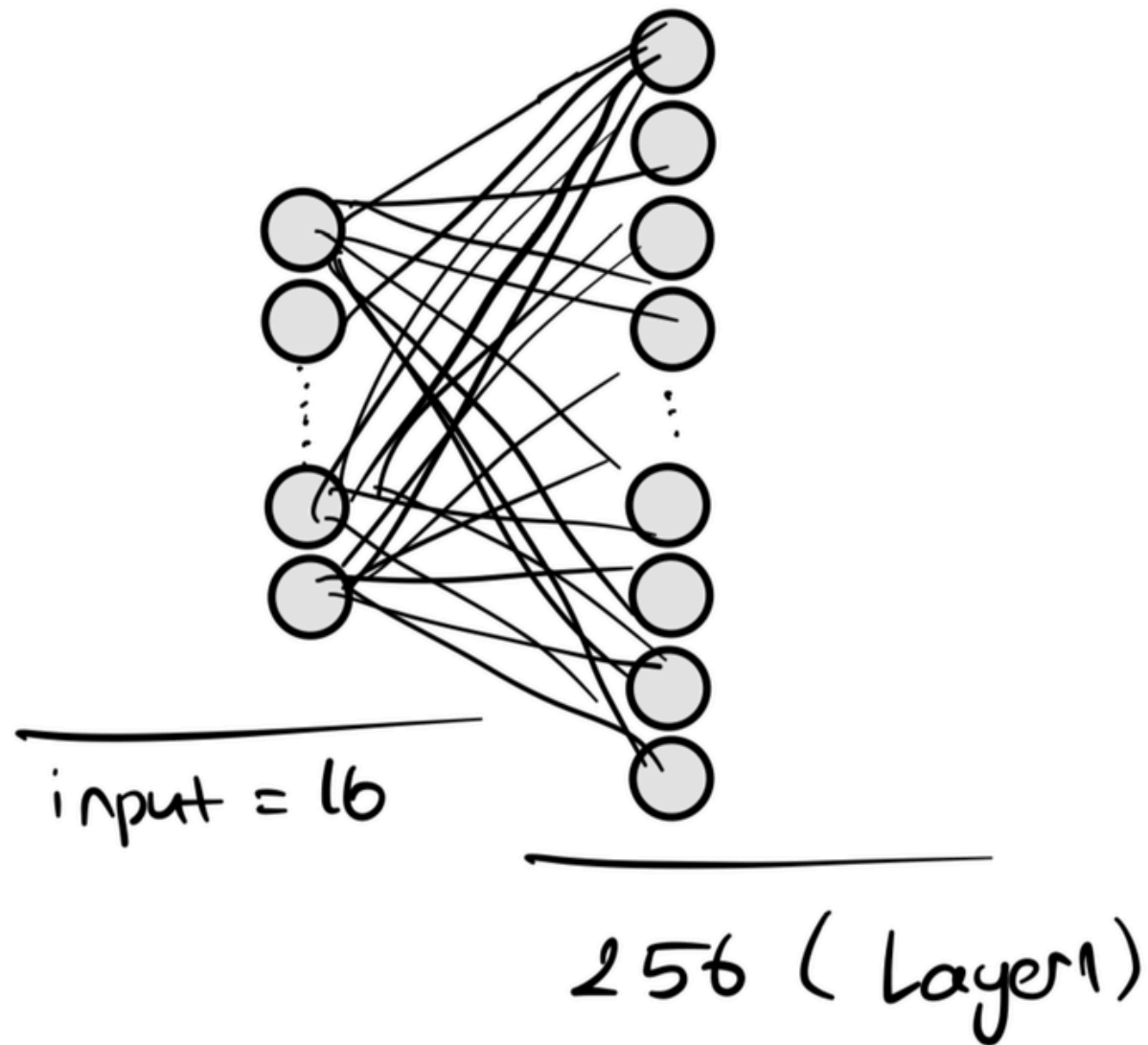
Reason2: They explode with input size



The hidden dim for the first layer is 256
input layer dim is 16

Is there a reason not use MLPs for vision ?

Reason2: They explode with input size



The hidden dim for the first layer is 256
input layer dim is 16

Then $w_1 = 16 \times 256 = 4096$ weight

Is there a reason not use MLPs for vision ?

Reason2: They explode with input size

That was only for gray scale 4x4 image.

If we calculate the weights for CIFAR images:

Is there a reason not use MLPs for vision ?

Reason2: They explode with input size

That was only for gray scale 4x4 image.

If we calculate the weights for CIFAR images:

$$w_1 = \underbrace{32 \times 32 \times 3}_{\text{flattened image size}} \times 256 = 787,712$$

Is there a reason not use MLPs for vision ?

Reason2: They explode with input size

That was only for gray scale 4x4 image.

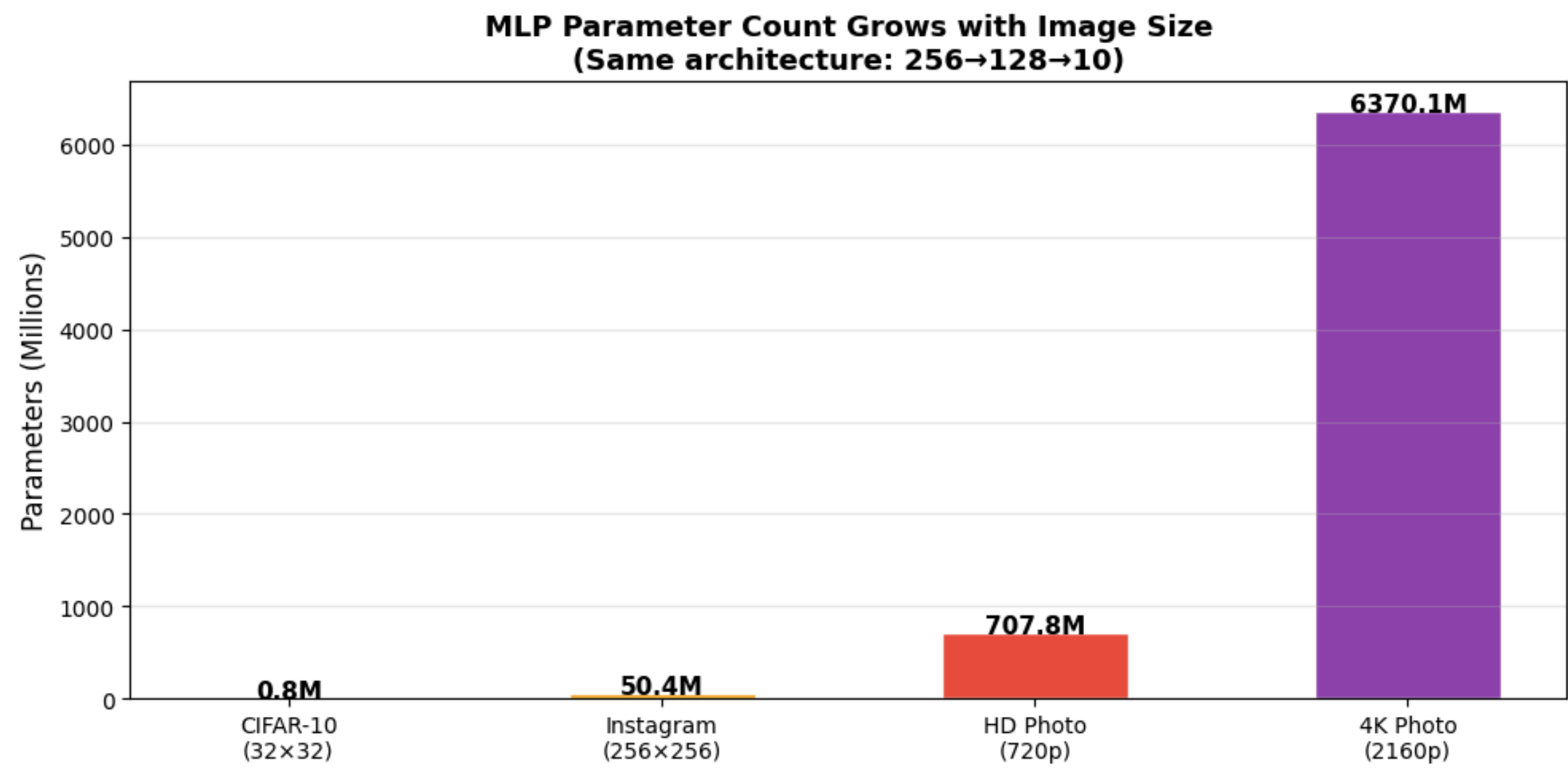
If we calculate the weights for CIFAR images:

$$w_1 = \underbrace{32 \times 32 \times 3}_{\text{flattened image size}} \times 256 = 787,712$$

<u>Image size</u>	<u>weights</u>
4 x 4 (gray image)	4096
32 x 32 (colored image)	787,712

Is there a reason not use MLPs for vision ?

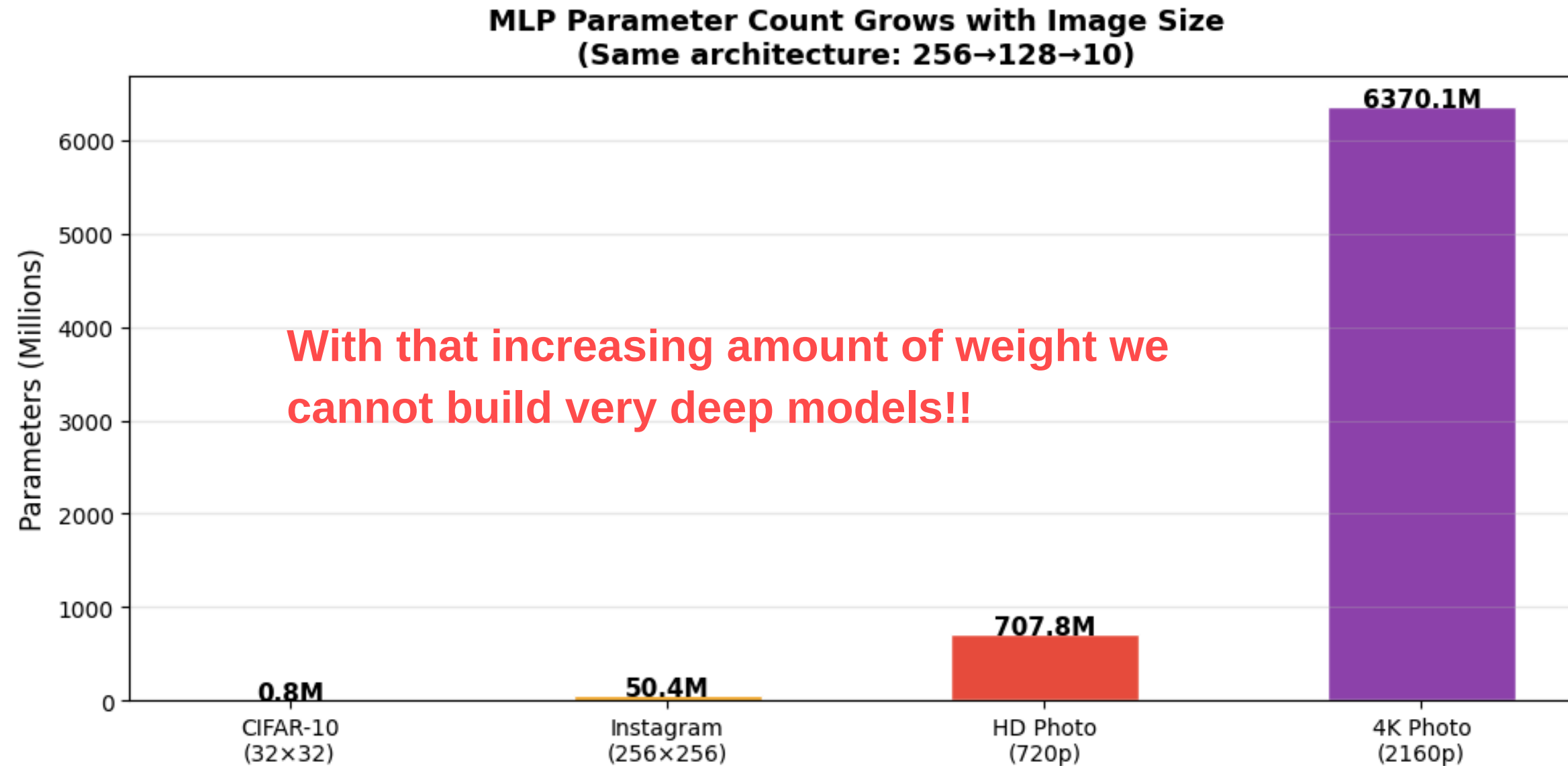
Reason2: They explode with input size



CIFAR-10 (32×32)	820,874 parameters
Instagram (256×256)	50,366,090 parameters
HD Photo (720p)	707,823,242 parameters
4K Photo (2160p)	6,370,133,642 parameters

Is there a reason not use MLPs for vision ?

Reason2: They explode with input size



CIFAR-10 (32×32)

820,874 parameters

Instagram (256×256)

50,366,090 parameters

HD Photo (720p)

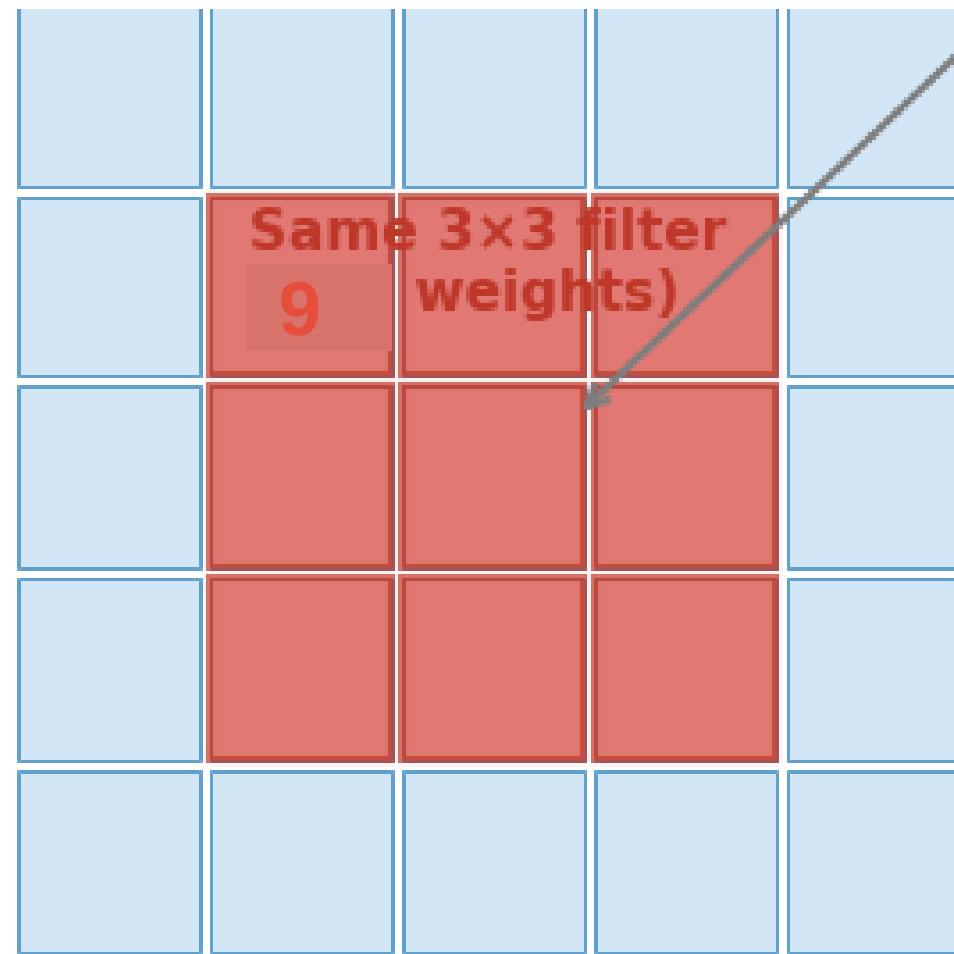
707,823,242 parameters

4K Photo (2160p)

6,370,133,642 parameters

How CNNs overcome this ?

Solution2: CNNs do not explode with the image size because they use fixed-size kernel (e.g. 3×3) that slides across the image. So a 3×3 filter has 9 weights whether the image is 32×32 or 512×512 .



How CNNs overcome this ?

Solution2: CNNs do not explode with the image size because they use fixed-size kernel (e.g. 3×3) that slides across the image. So a 3×3 filter has 9 weights whether the image is 32×32 or 512×512 .

Define filter (3×3):

$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

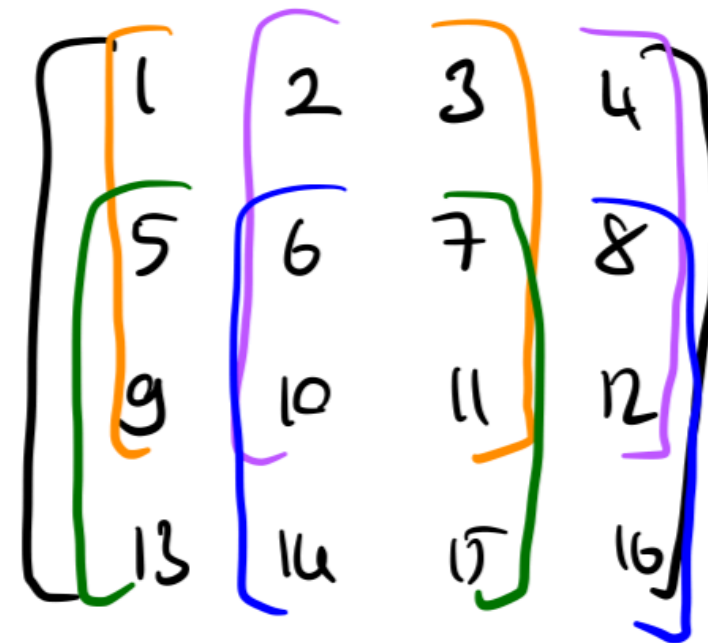
How CNNs overcome this ?

Solution2: CNNs do not explode with the image size because they use fixed-size kernel (e.g. 3×3) that slides across the image. So a 3×3 filter has 9 weights whether the image is 32×32 or 512×512 .

Define filter (3×3):

$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

Extract every 3×3 patch from image



How CNNs overcome this ?

Solution2: CNNs do not explode with the image size because they use fixed-size kernel (e.g. 3×3) that slides across the image. So a 3×3 filter has 9 weights whether the image is 32×32 or 512×512 .

Position (0,0)

$$\begin{bmatrix} 1 & 2 & 3 \\ 5 & 6 & 7 \\ 9 & 10 & 11 \end{bmatrix}$$

Position (0,1)

$$\begin{bmatrix} 2 & 3 & 4 \\ 6 & 7 & 8 \\ 10 & 11 & 12 \end{bmatrix}$$

Position (1,0)

$$\begin{bmatrix} 5 & 6 & 7 \\ 9 & 10 & 11 \\ 13 & 14 & 15 \end{bmatrix}$$

Position (1,1)

$$\begin{bmatrix} 6 & 7 & 8 \\ 10 & 11 & 12 \\ 14 & 15 & 16 \end{bmatrix}$$

How CNNs overcome this ?

Solution2: CNNs do not explode with the image size because they use fixed-size kernel (e.g. 3×3) that slides across the image. So a 3×3 filter has 9 weights whether the image is 32×32 or 512×512 .

Position (0,0)

$$\begin{bmatrix} 1 & 2 & 3 \\ 5 & 6 & 7 \\ 9 & 10 & 11 \end{bmatrix}$$

Position (0,1)

$$\begin{bmatrix} 2 & 3 & 4 \\ 6 & 7 & 8 \\ 10 & 11 & 12 \end{bmatrix}$$

We apply the same filter at each position.

Patch

$$\begin{bmatrix} 2 & 3 & 4 \\ 6 & 7 & 8 \\ 10 & 11 & 12 \end{bmatrix}$$

Filter

$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

Dot product

•

$$= 2 + 0 - 4 + 12 + 0 - 16 + 10 + 0 - 12 = -8$$

Position (1,0)

$$\begin{bmatrix} 5 & 6 & 7 \\ 9 & 10 & 11 \\ 13 & 14 & 15 \end{bmatrix}$$

Position (1,1)

$$\begin{bmatrix} 6 & 7 & 8 \\ 10 & 11 & 12 \\ 14 & 15 & 16 \end{bmatrix}$$

How CNNs overcome this ?

Solution2: CNNs do not explode with the image size because they use fixed-size kernel (e.g. 3×3) that slides across the image. So a 3×3 filter has 9 weights whether the image is 32×32 or 512×512 .

Position (0,0)

$$\begin{bmatrix} 1 & 2 & 3 \\ 5 & 6 & 7 \\ 9 & 10 & 11 \end{bmatrix}$$

Position (0,1)

$$\begin{bmatrix} 2 & 3 & 4 \\ 6 & 7 & 8 \\ 10 & 11 & 12 \end{bmatrix}$$

We apply the same filter at each position.

Patch

$$\begin{bmatrix} 2 & 3 & 4 \\ 6 & 7 & 8 \\ 10 & 11 & 12 \end{bmatrix}$$

Filter

$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

Dot product

•

$$= 2 + 0 - 4 + 12 + 0 - 16 + 10 + 0 - 12 = -8$$

Position (1,0)

$$\begin{bmatrix} 5 & 6 & 7 \\ 9 & 10 & 11 \\ 13 & 14 & 15 \end{bmatrix}$$

Position (1,1)

$$\begin{bmatrix} 6 & 7 & 8 \\ 10 & 11 & 12 \\ 14 & 15 & 16 \end{bmatrix}$$

How CNNs overcome this ?

Solution2: CNNs do not explode with the image size because they use fixed-size kernel (e.g. 3×3) that slides across the image. So a 3×3 filter has 9 weights whether the image is 32×32 or 512×512 .

The same 9 weight used 4 times.

The weight size is independent from the image size.

If we have bigger image with exact same filter size we will have more positions to apply the filter.

Number of iteration (multiplication) increase
but it is independent from the weight size.

How CNNs overcome this ?

Solution2: CNNs do not explode with the image size because they use fixed-size kernel (e.g. 3×3) that slides across the image. So a 3×3 filter has 9 weights whether the image is 32×32 or 512×512 .

It depends on the number of filters you use.

In practice we use 32, 64, 128, 512 filters.

If we use 32 filters.

Each filter = $3 \times 3 \times 1 = 9$ weights

32 filters = $3 \times 3 \times 32 = 288$ weights.

How CNNs overcome this ?

Solution2: CNNs do not explode with the image size because they use fixed-size kernel (e.g. 3×3) that slides across the image. So a 3×3 filter has 9 weights whether the image is 32×32 or 512×512 .

For the same input size $L \times L$ image MLP required $\rightarrow 4096$ 🙄
CNN required $\rightarrow 288$ 😊

Is there a reason not use MLPs for vision ?

Reason3: Not translation equivariant and invariant

Equivariance: if you transform the input, the representation transforms in a predictable, corresponding way.

If you shift an image right by 10px, the feature map also shifts right by 10px and the structure is preserved.

Invariance: The output remains the same regardless of where the input appears.

If you shift an image right by 10px, the model still predict the class correctly.

But Invariance translation also a bit problematic with cnns as well. So that is why we use data augmentation.

How CNNs overcome this ?

Solution3: Spatial structure is preserved layer by layer. Nearby pixels stay "nearby" in the feature map no matter the translation.

The convolutional layers in CNN architecture provides a model equivariance and combination of cnn with max pooling leads to translation invariance.

Interview Moment: CNN Only Talks to Its Neighbors



You're an computer vision engineer at a manufacturing company.

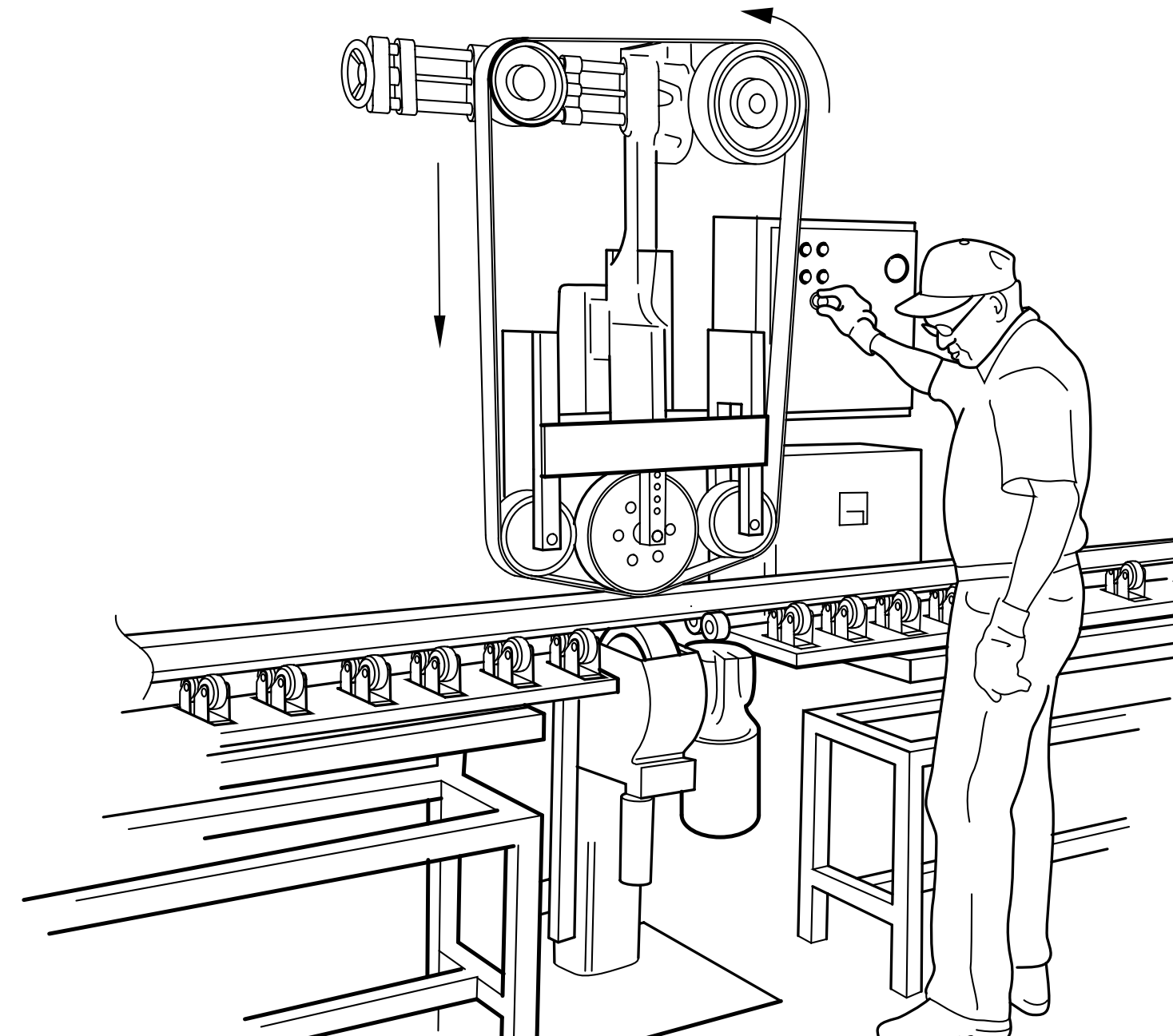
You build an **AI model** to **detect defects** on the product.

You have built the **deepest CNN** you can.

The model performs **well** on **detecting local defects**.

But on a new quality control task where the **model needs** to **reason** about the **relationship** between a **crack** in one **corner** and **material stress** patterns in the **opposite corner** your deep **CNN** still **underperforms** despite all the **depth** and **compute**.

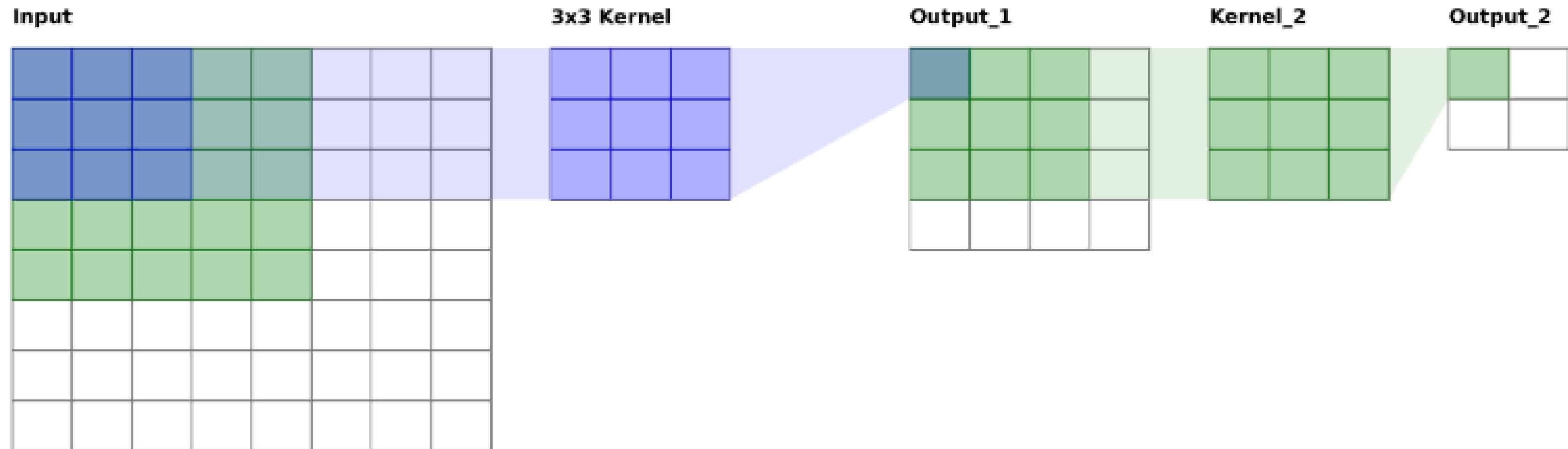
What is the **architectural ceiling** we are **hitting** in that case?



Is CNN is the best approach we can use for vision ?

CNN architectures were used a lot in the field and still used for vision. But after Attention paper in 2017 the transformers specifically vision transformers also became a major part in the computer vision.

Reason: CNNs look at the 'local' receptive fields.



Is CNN is the best approach we can use for vision ?

CNN architectures were used a lot in the field and still used for vision. But after Attention paper in 2017 the transformers specifically vision transformers also became a major part in the computer vision.

Reason: CNNs look at the 'local' receptive fields.

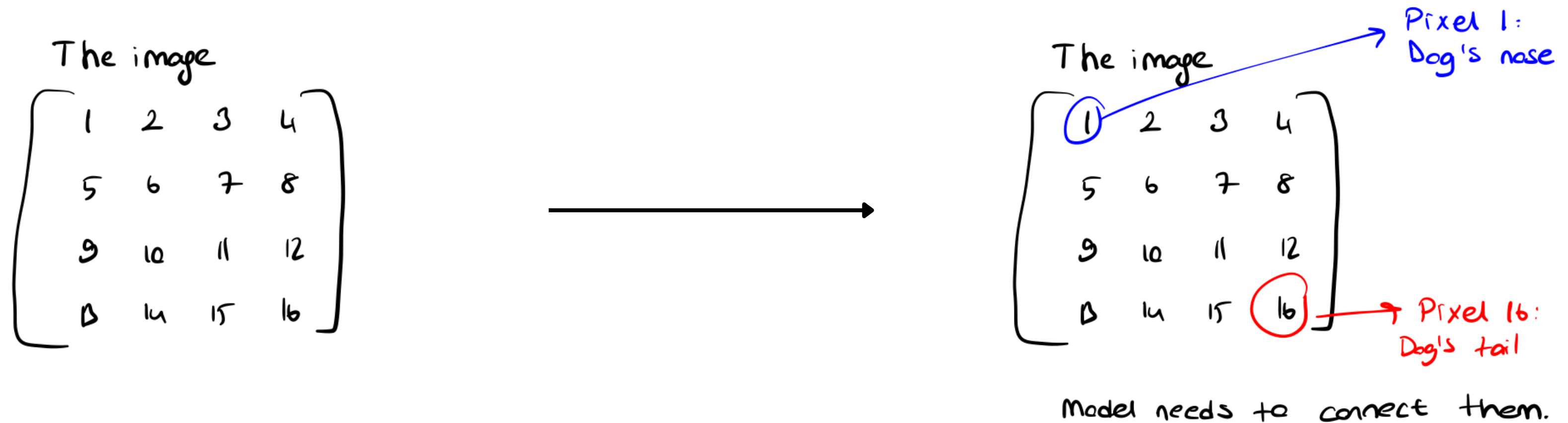
The image

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Is CNN is the best approach we can use for vision ?

CNN architectures were used a lot in the field and still used for vision. But after Attention paper in 2017 the transformers specifically vision transformers also became a major part in the computer vision.

Reason: CNNs look at the 'local' receptive fields.



Is CNN is the best approach we can use for vision ?

CNN architectures were used a lot in the field and still used for vision. But after Attention paper in 2017 the transformers specifically vision transformers also became a major part in the computer vision.

Reason: CNNs look at the 'local' receptive fields.

We don't want to shrink dimension so we apply zero-padding first.

0	0	0	0	0	0
0	1	2	3	4	0
0	5	6	7	8	0
0	9	10	11	12	0
0	13	14	15	16	0
0	0	0	0	0	0

Padded image.

Is CNN is the best approach we can use for vision ?

CNN architectures were used a lot in the field and still used for vision. But after Attention paper in 2017 the transformers specifically vision transformers also became a major part in the computer vision.

Reason: CNNs look at the 'local' receptive fields.

We don't want to shrink dimension so we apply zero-padding first.

0	0	0	0	0	0
0	1	2	3	4	0
0	5	6	7	8	0
0	9	10	11	12	0
0	13	14	15	16	0
0	0	0	0	0	0

Padded image.



0	0	0	0	0	0
0	1	2	3	4	0
0	5	6	7	8	0
0	9	10	11	12	0
0	13	14	15	16	0
0	0	0	0	0	0

At position (0,0)
kernel sees this

Is CNN is the best approach we can use for vision ?

CNN architectures were used a lot in the field and still used for vision. But after Attention paper in 2017 the transformers specifically vision transformers also became a major part in the computer vision.

Reason: CNNs look at the 'local' receptive fields.

We don't want to shrink dimension so we apply zero-padding first.

0	0	0	0	0	0
0	1	2	3	4	0
0	5	6	7	8	0
0	9	10	11	12	0
0	13	14	15	16	0
0	0	0	0	0	0

Padded image.



0	0	0	0	0	0
0	1	2	3	4	0
0	5	6	7	8	0
0	9	10	11	12	0
0	13	14	15	16	0
0	0	0	0	0	0

At position (0,0)
kernel sees this



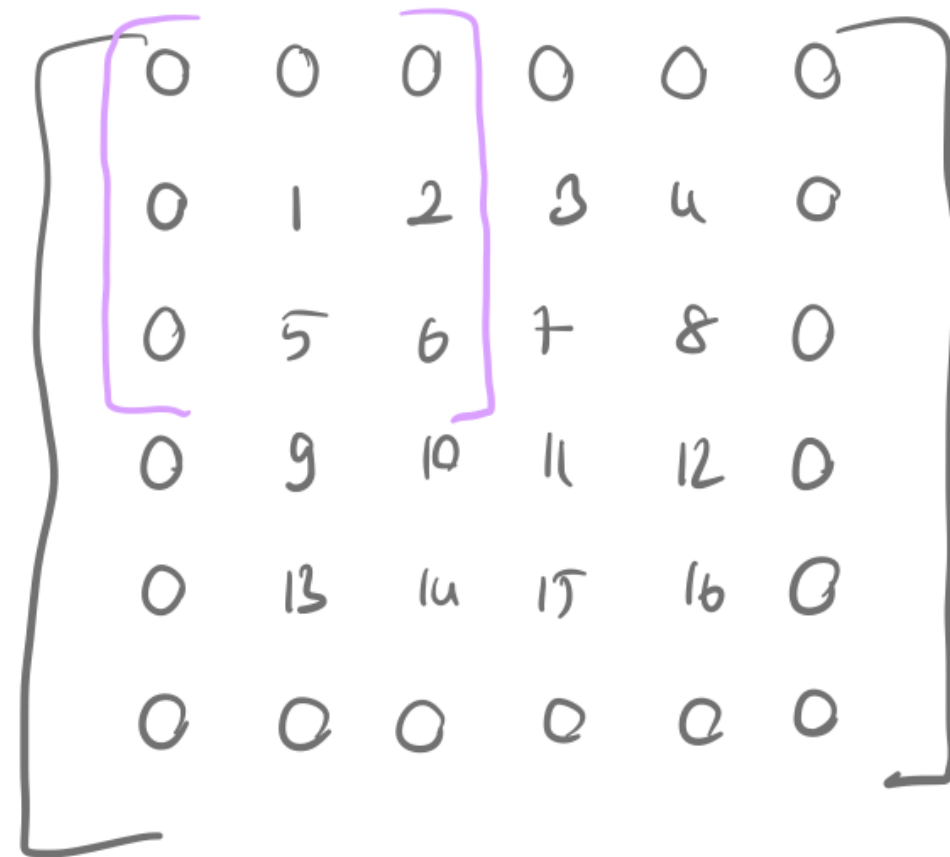
0	0	0
0	1	2
0	5	6

Pixel 16
invisible!

Is CNN is the best approach we can use for vision ?

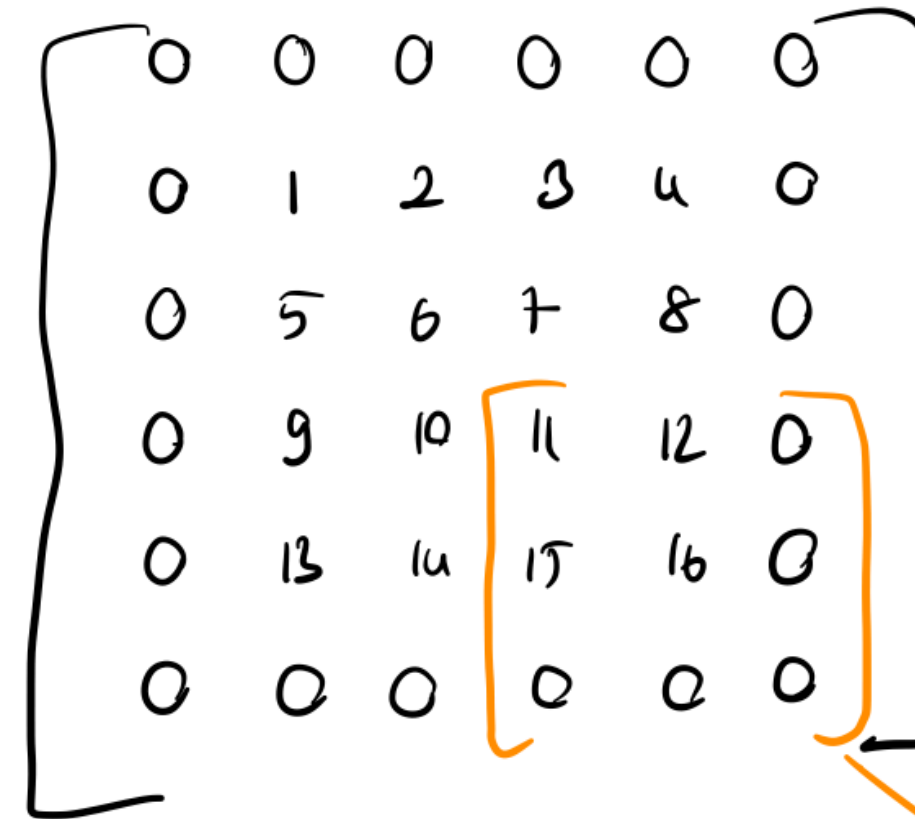
CNN architectures were used a lot in the field and still used for vision. But after Attention paper in 2017 the transformers specifically vision transformers also became a major part in the computer vision.

Reason: CNNs look at the 'local' receptive fields.



At position (0,0)

kernel sees this



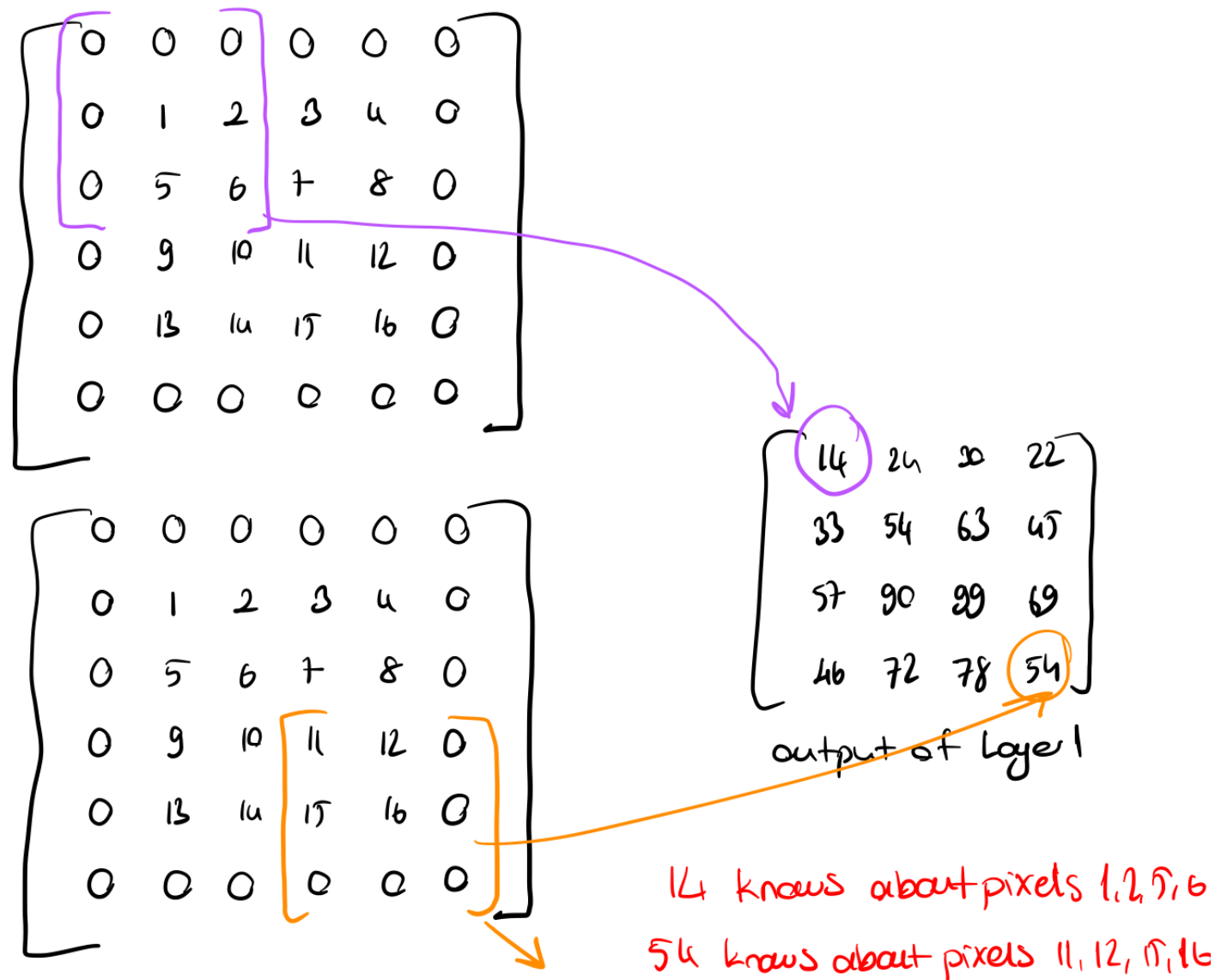
At position (3,3) kernel sees

No pixel + information!

Is CNN is the best approach we can use for vision ?

CNN architectures were used a lot in the field and still used for vision. But after Attention paper in 2017 the transformers specifically vision transformers also became a major part in the computer vision.

Reason: CNNs look at the 'local' receptive fields.



If you look at every single value at the output you see that there is no single value that has seen pixel 1 and pixel 16 at the same time !

AND they don't get interacted until later layers.

What is the main motivation behind transformers to use in vision?

Solution: In ViTs we have **attention** mechanism.

We split the image 2×2 patches $\rightarrow$ 4 tokens

$$T_1 = [1, 2, 5, 6]$$

$$T_2 = [3, 4, 7, 8]$$

$$T_3 = [9, 10, 13, 14]$$

$$T_4 = [11, 12, 15, 16]$$

What is the main motivation behind transformers to use in vision?

Solution: In ViTs we have **attention** mechanism.

We split the image 2×2 patches $\rightarrow$ 4 tokens

$$T_1 = [1, 2, 5, 6]$$

$$T_2 = [3, 4, 7, 8]$$

$$T_3 = [9, 10, 13, 14]$$

$$T_4 = [11, 12, 15, 16]$$

ViT computes attention between all patches

For simplicity we compute attention as:

$$\text{Score}(T_i, T_j) = T_i \cdot T_j$$

What is the main motivation behind transformers to use in vision?

Solution: In ViTs we have **attention** mechanism.

We split the image 2×2 patches $\rightarrow$ 4 tokens

$$T_1 = [1, 2, 5, 6]$$

$$T_2 = [3, 4, 7, 8]$$

$$T_3 = [9, 10, 13, 14]$$

$$T_4 = [11, 12, 15, 16]$$

So:

$$T_1 \cdot T_1 = 1 \cdot 1 + 2 \cdot 2 + 5 \cdot 5 + 6 \cdot 6 = 66$$

$$T_1 \cdot T_2 = 94$$

$$T_1 \cdot T_3 = 178$$

$$T_1 \cdot T_4 = 206$$

$\rightarrow$ pixel 1 and pixel 16 is connected
from the beginning!

$\vdots$

$$T_2 \cdot T_3 = 270$$

$$T_2 \cdot T_4 = \vdots$$

ViT computes attention between all patches

For simplicity we compute attention as:

$$\text{Score}(T_i, T_j) = T_i \cdot T_j$$

What is the main motivation behind transformers to use in vision?

Solution: In ViTs we have **attention** mechanism.

Attention scores between patches

	T_1	T_2	T_3	T_4
T_1	66	94	178	206
T_2	94	138	270	314
T_3	178	270	546	638
T_4	206	314	638	749

What is the main motivation behind transformers to use in vision?

Solution: In ViTs we have **attention** mechanism.

Attention scores between patches

	T_1	T_2	T_3	T_4
T_1	66	94	178	206
T_2	94	138	270	314
T_3	178	270	546	638
T_4	206	314	638	749



How much patch i "looks at" patch j .

What is the main motivation behind transformers to use in vision?

Solution: In ViTs we have **attention** mechanism.

Attention scores between patches

	T_1	T_2	T_3	T_4
T_1	66	94	178	206
T_2	94	138	270	314
T_3	178	270	546	638
T_4	206	314	638	749

How much patch i "looks at" patch j .

In the actual ViT we use this formula:

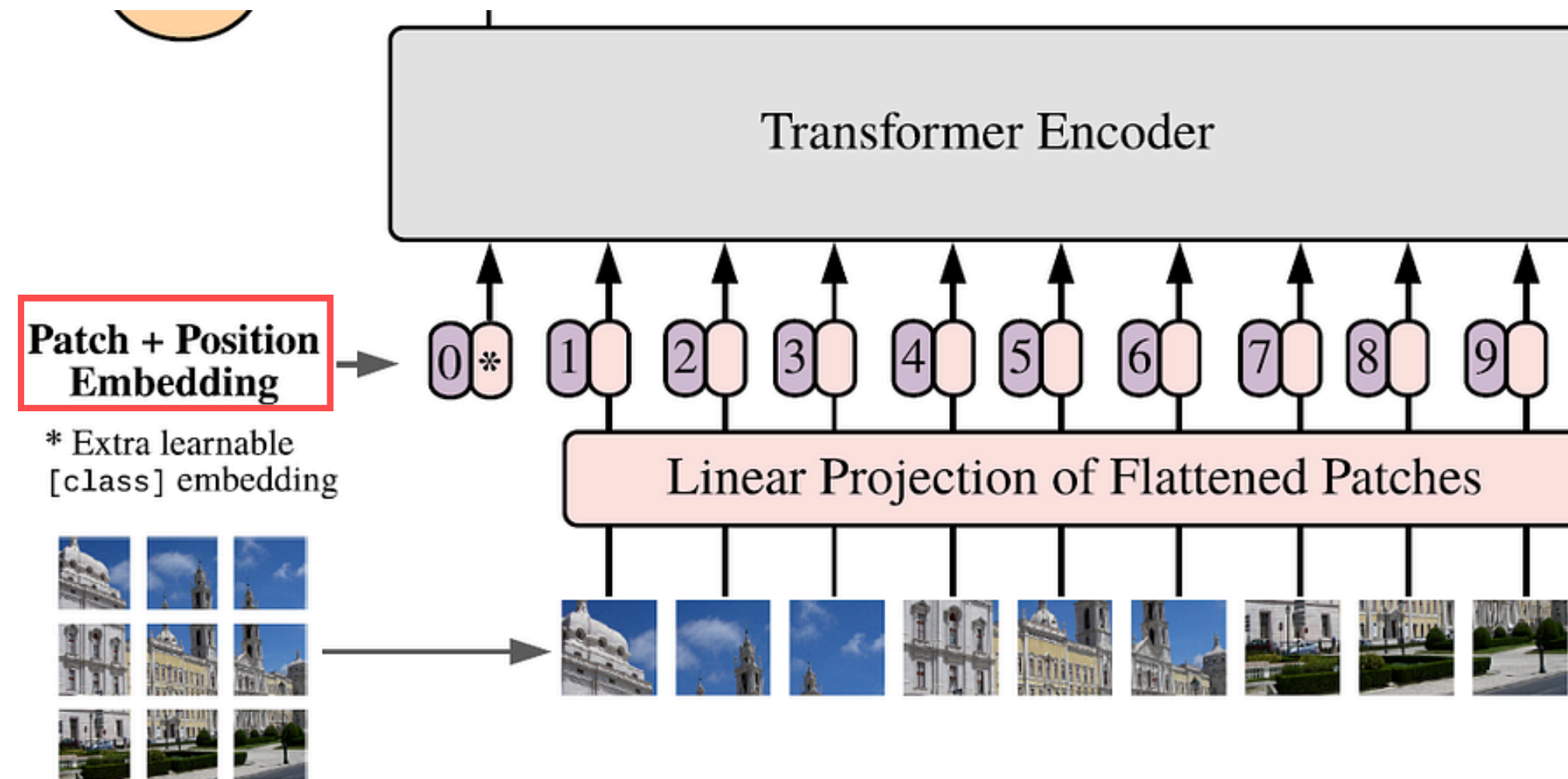
$$\text{Score}(T_i, T_j) = (T_i \cdot W_Q) \cdot (T_j \cdot W_K)^T / \sqrt{d}$$

learnable weight parameters

embedding dimension.

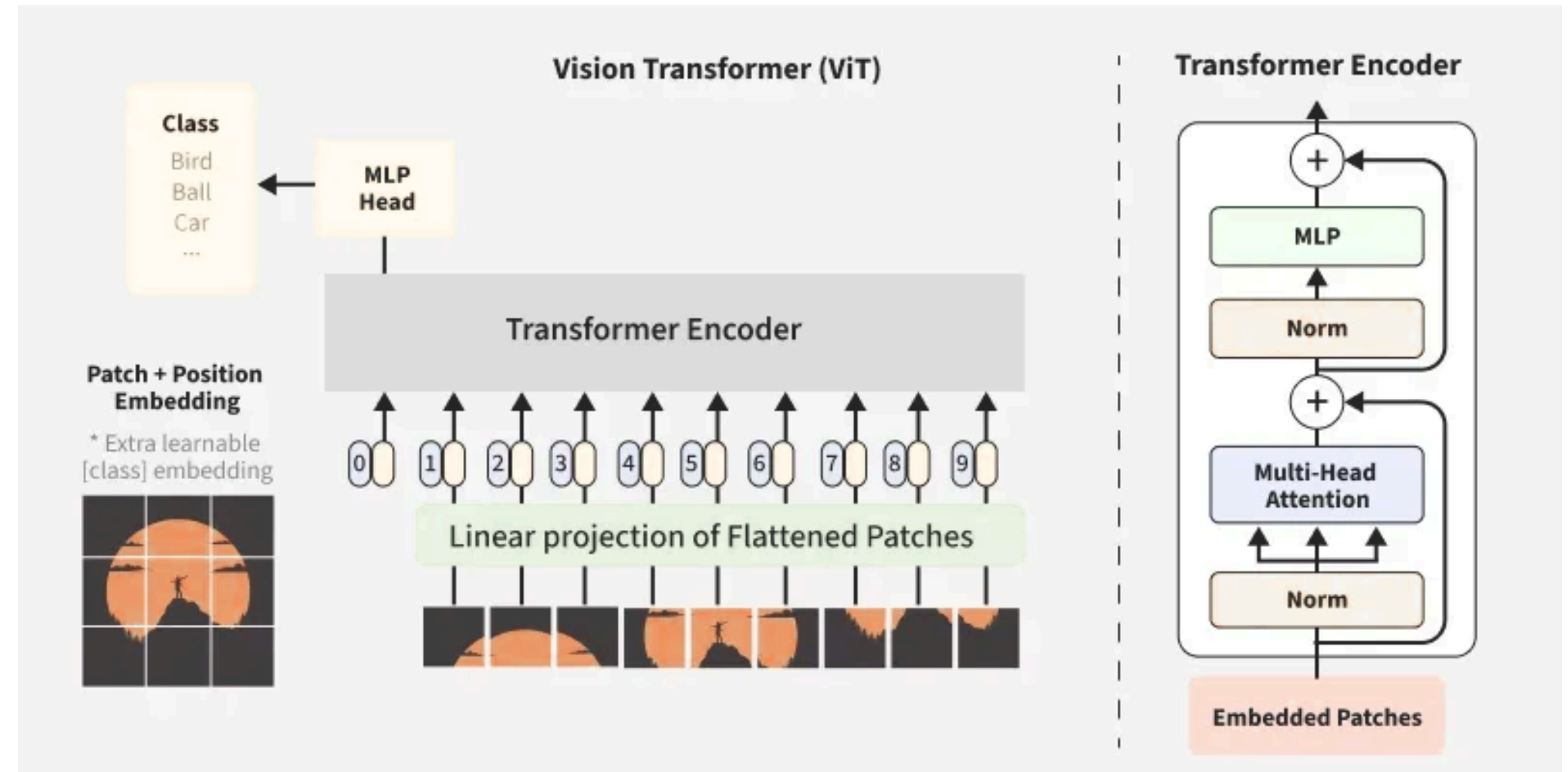
But then how ViT knows the location of each patch?

Transformers can solve this problem because they have position encoding in the beginning.



What makes ViTs good?

- They have global receptive field
- There is an attention mechanism
- They have position encoding



Interview Moment: The Great Fine-Tuning Debate



You just joined a small AI startup as their first ML engineer.

They want to build a plant **disease detection** system for farmers in rural areas.

You have **1,500 labeled images** of diseased and healthy crops, a **single GPU**, and a **two week** deadline.

Your CTO points to a **pretrained ViT-B/16** trained on ImageNet and says 'just **fine-tune** the **whole thing**, it's state of the art.'

You have **serious doubts**.

What specific **risks** does **full fine-tuning** introduce in this situation, what strategy would you actually use, and how would you justify it to your CTO?



Interview Moment: The Great Fine-Tuning Debate



Full **fine-tuning** a **BIG** model with **only 1,500 images** is a classic recipe for **overfitting**. The model will almost certainly **overwrite** the rich **visual representations** built from ImageNet.

You end up with the **worst of both worlds**, **losing** pretrained **knowledge without** gaining **domain expertise**.

Depending on the **domain shift**, the **smarter move** is to **freeze** the **entire backbone** and only **train** the **classification** head. The **pretrained ViT** already **encodes** genuinely **useful** high-level **features** like textures, color gradients, edge patterns that could transfer to plant disease detection.

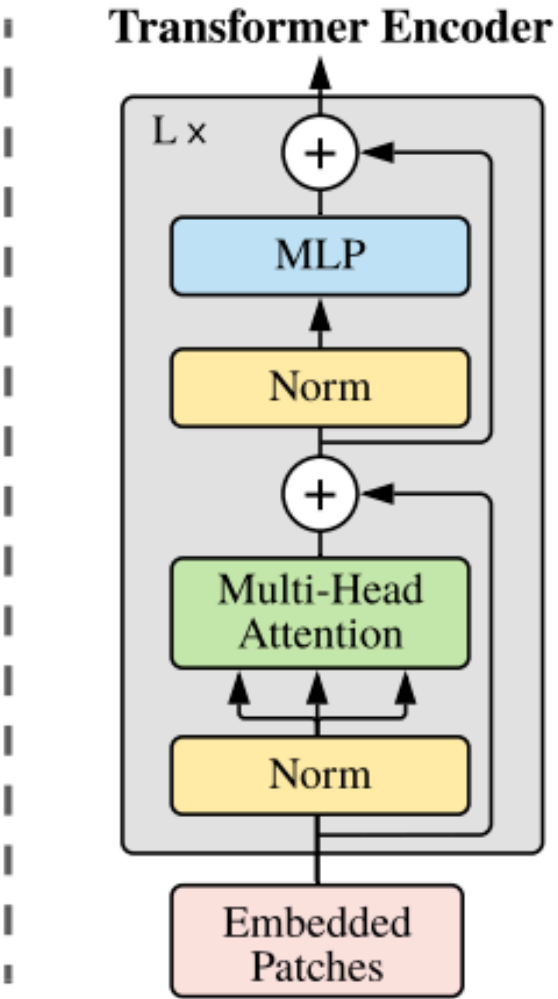
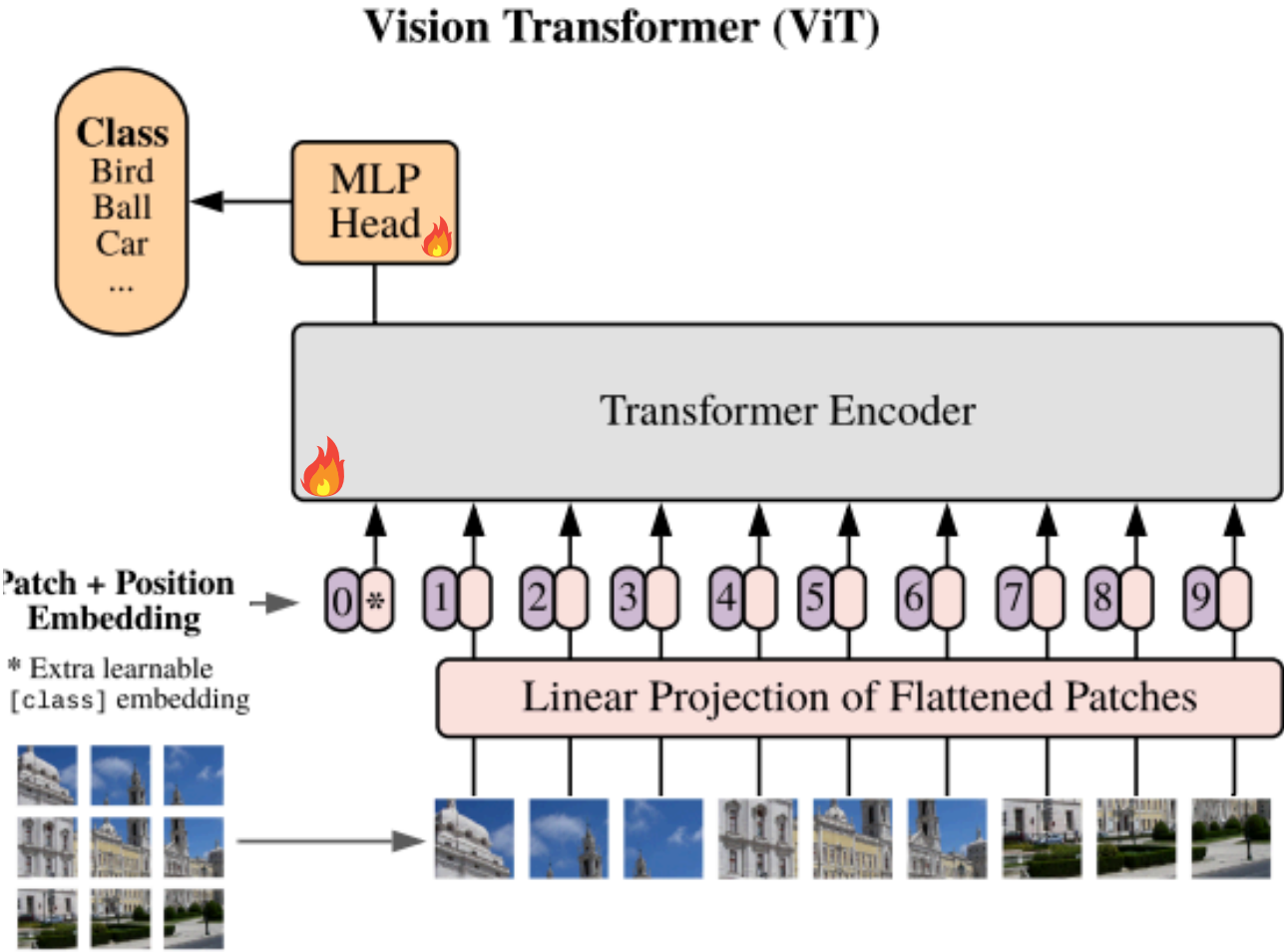
Freezing reduces trainable parameters from 86M to roughly 3,000, which dramatically cuts overfitting risk, GPU memory requirements, and training time simultaneously.

If head-only training plateaus, I would then gradually unfreeze the last one or two transformer blocks to allow slight domain adaptation. Or maybe I can still keep the whole architecture frozen and add additional layers (adapter, lora, prompt) to existing model and train them with the plant dataset that I have.



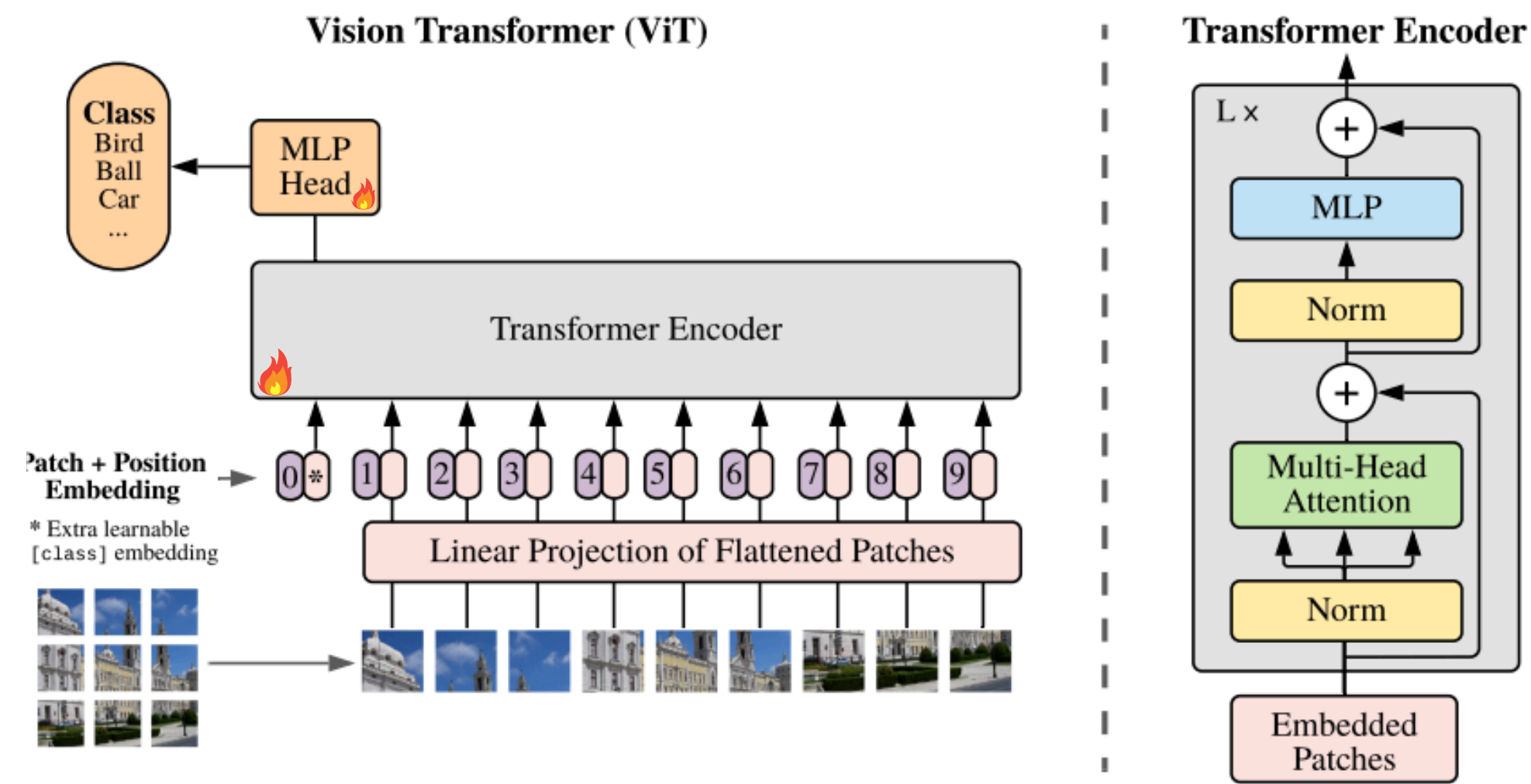
How do we use Transformers in practice?

	Starting point	Data needed	Who does this?
Train from scratch	Random weights	Massive	Google, Meta, OpenAI



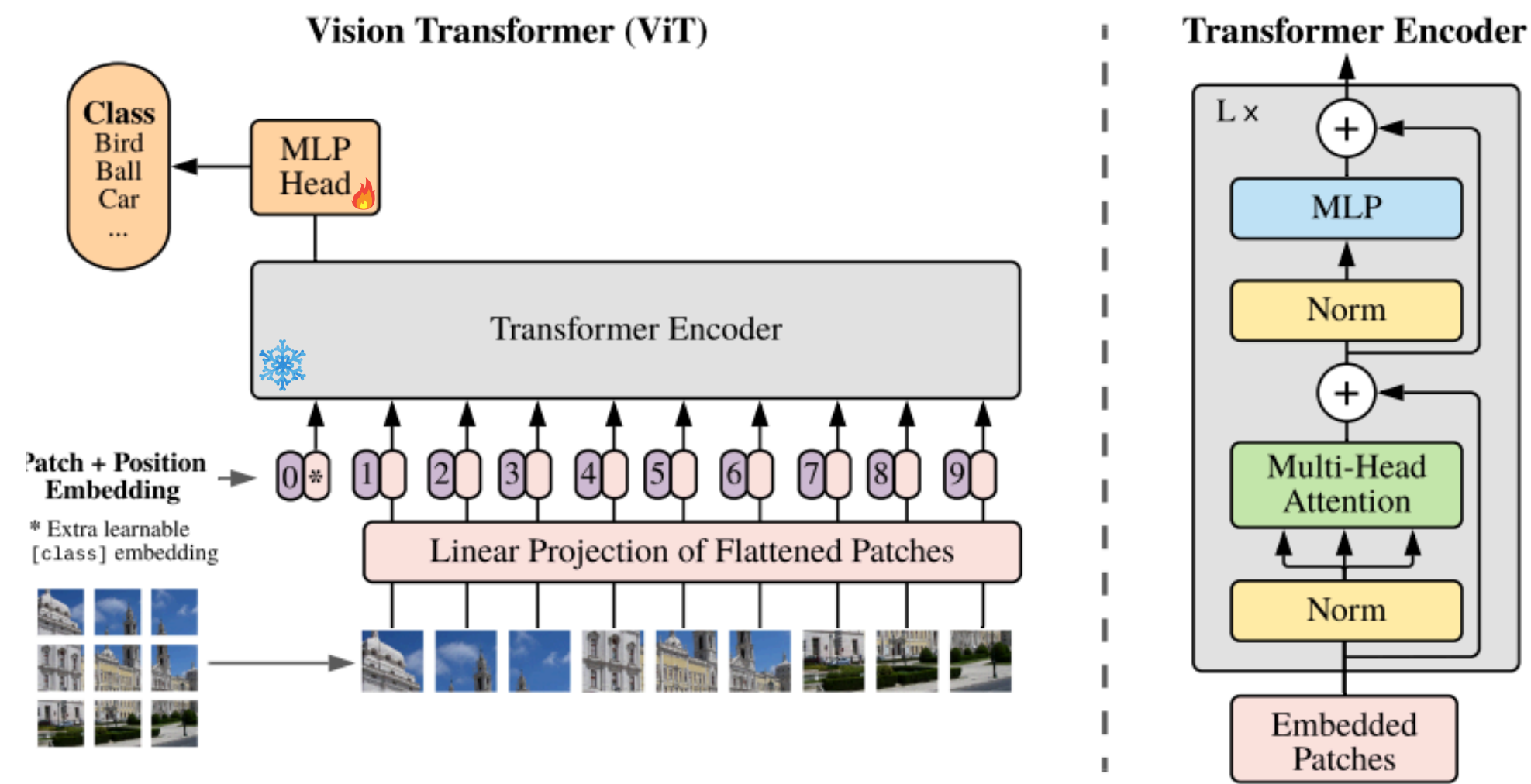
How do we use Transformers in practice?

	Starting point	Data needed	Who does this?
Train from scratch	Random weights	Massive	Google, Meta, OpenAI
Fine-tune all weights	Pretrained checkpoint	Medium (thousands–millions)	ML teams with GPUs



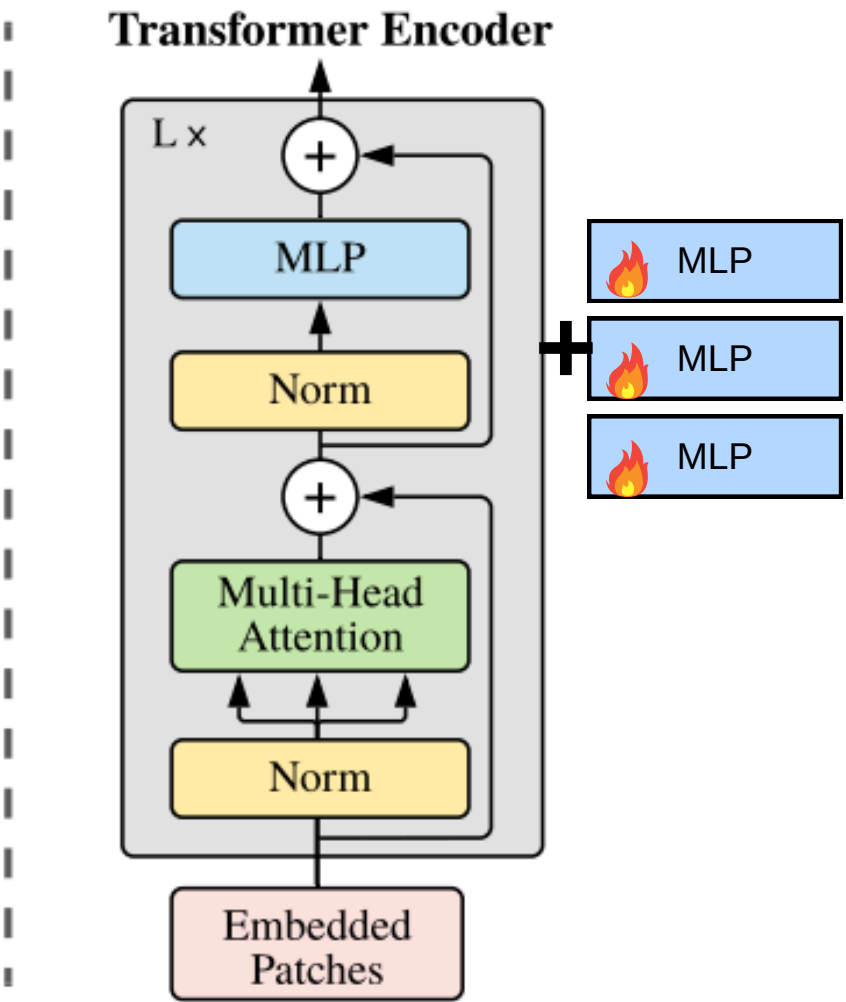
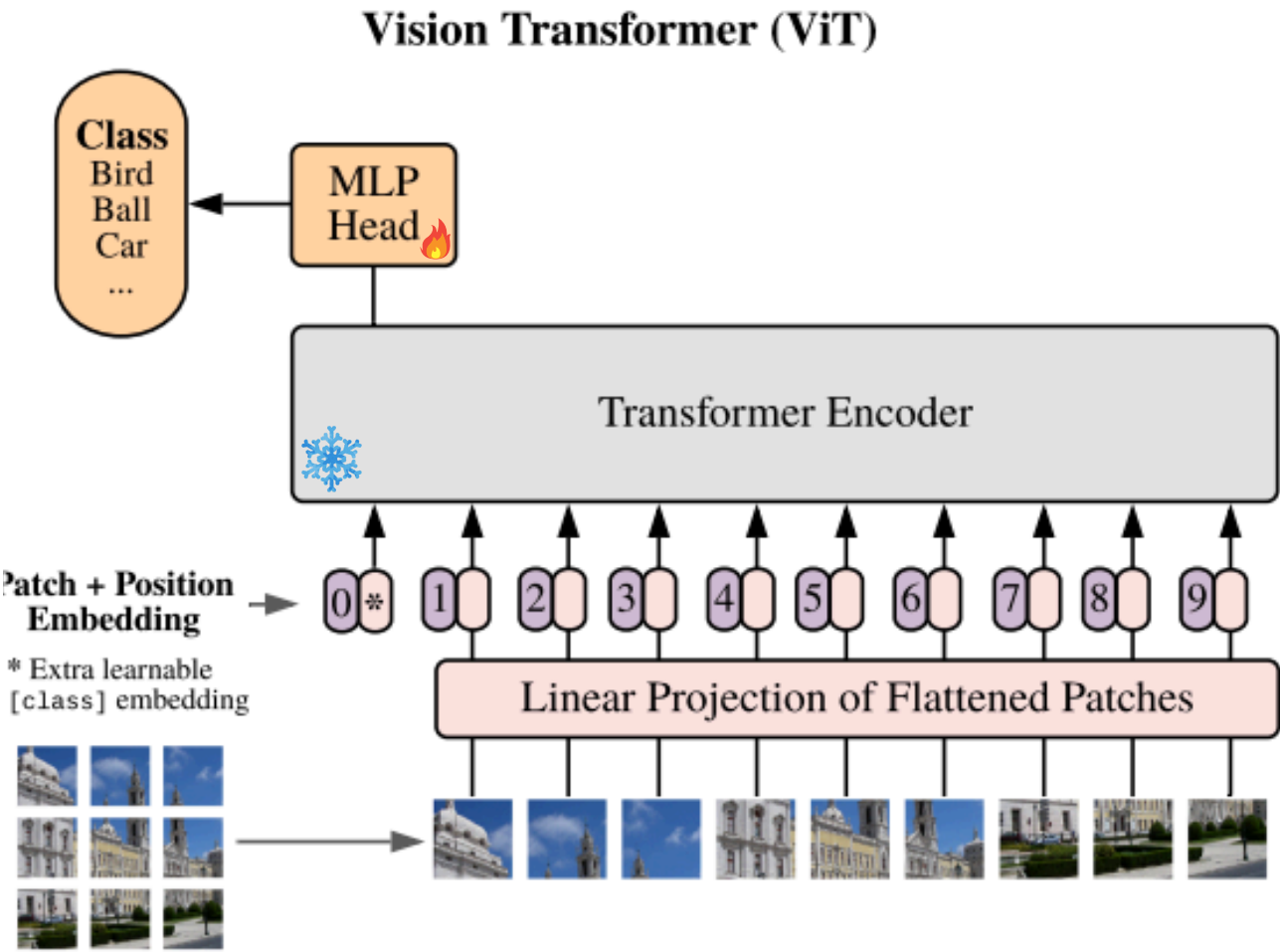
How do we use Transformers in practice?

	Starting point	Data needed	Who does this?
Train from scratch	Random weights	Massive	Google, Meta, OpenAI
Fine-tune all weights	Pretrained checkpoint	Medium (thousands–millions)	ML teams with GPUs
Fine-tune head only	Pretrained checkpoint	Small (hundreds–thousands)	Anyone



How do we use Transformers in practice?

	Starting point	Data needed	Who does this?
Train from scratch	Random weights	Massive	Google, Meta, OpenAI
Fine-tune all weights	Pretrained checkpoint	Medium (thousands–millions)	ML teams with GPUs
Fine-tune head only	Pretrained checkpoint	Small (hundreds–thousands)	Anyone
PEFT (LoRA, adapters)	Pretrained checkpoint	Small–Medium	Anyone



Now is your turn

You can work in Teams

Total given time: 60 minutes

You need to do analysis as well and report the best model parameters

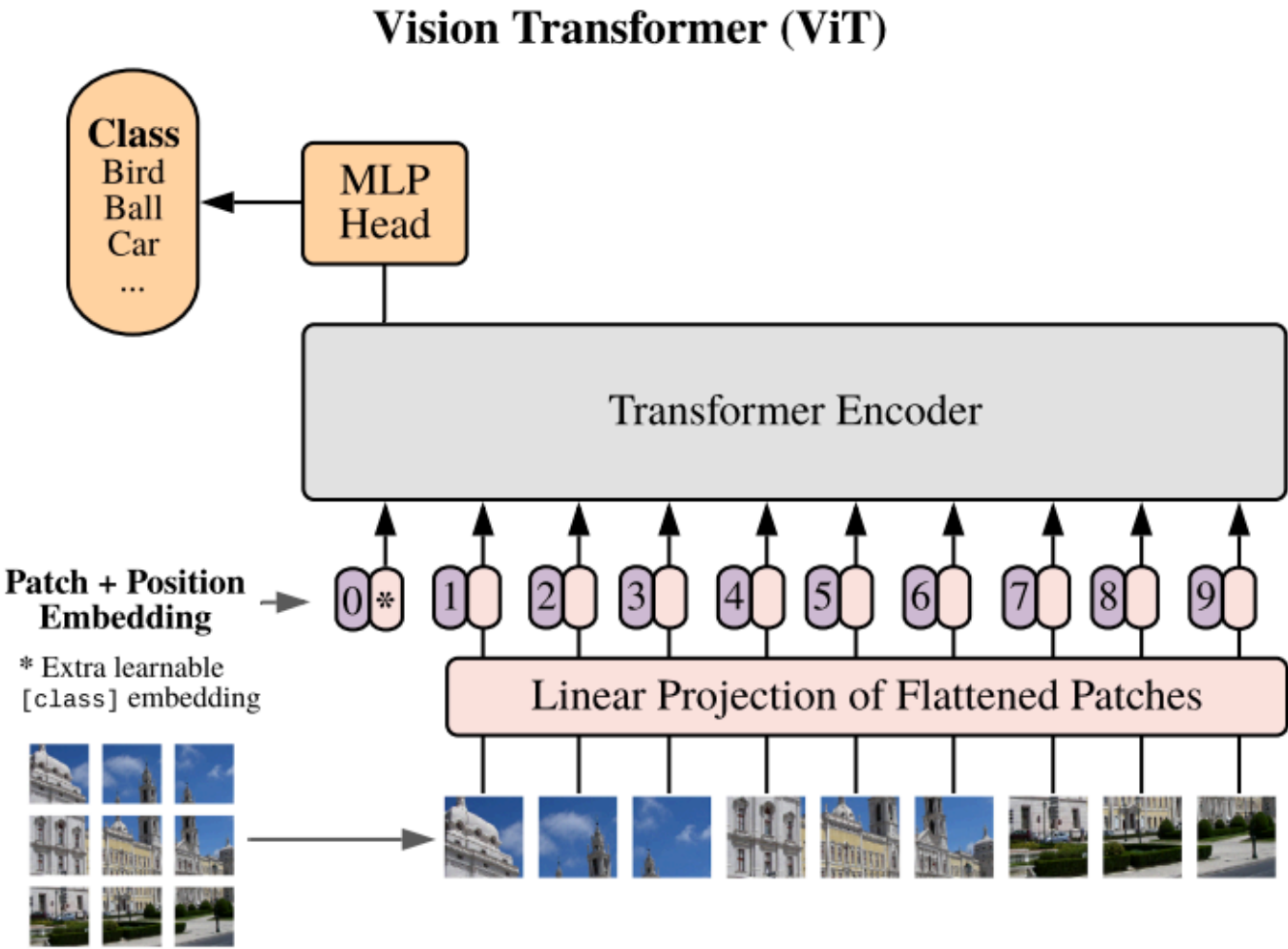
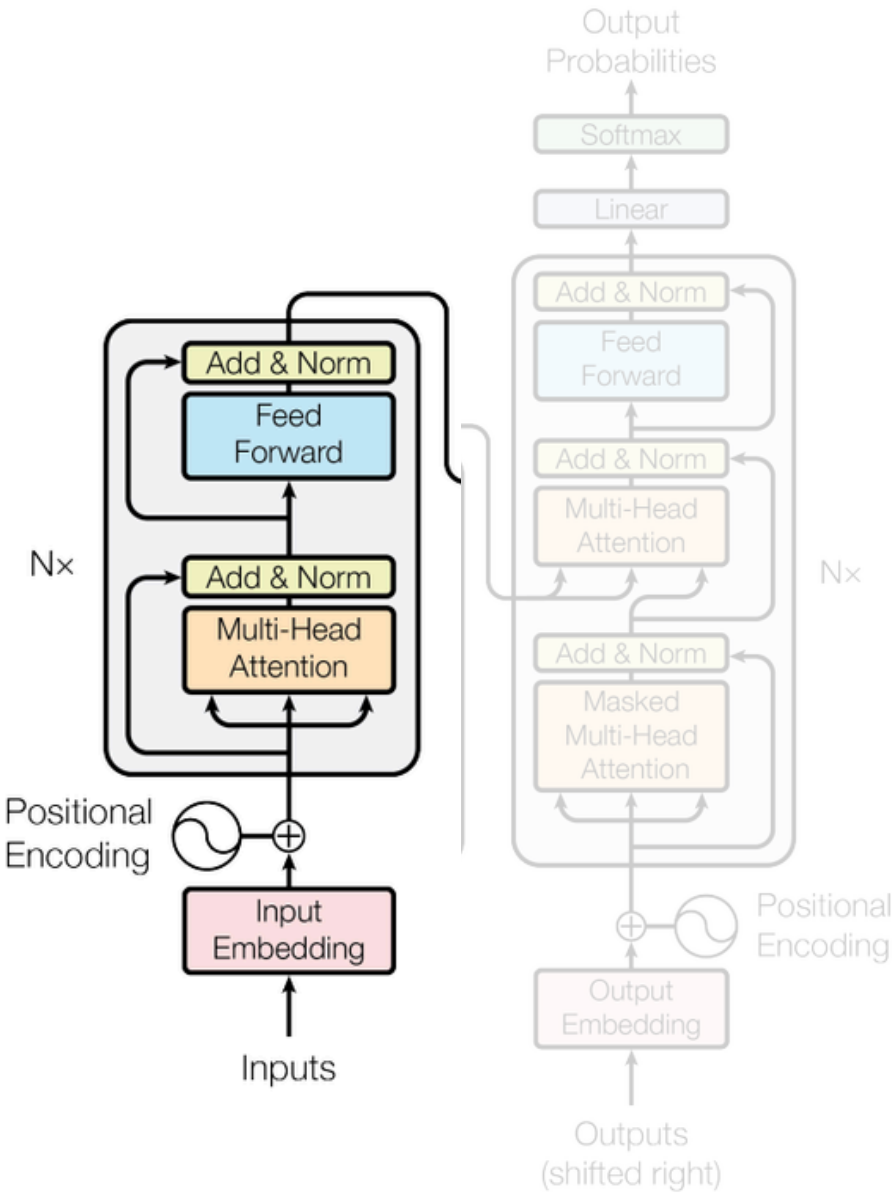
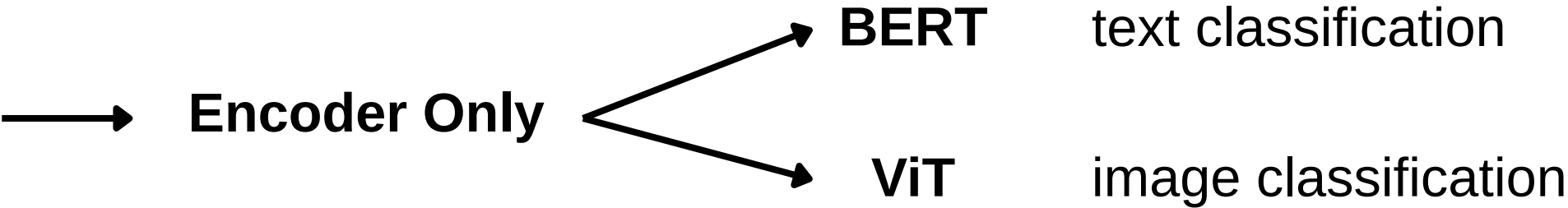
Your feedback for these lab sessions



How to select which model to use?

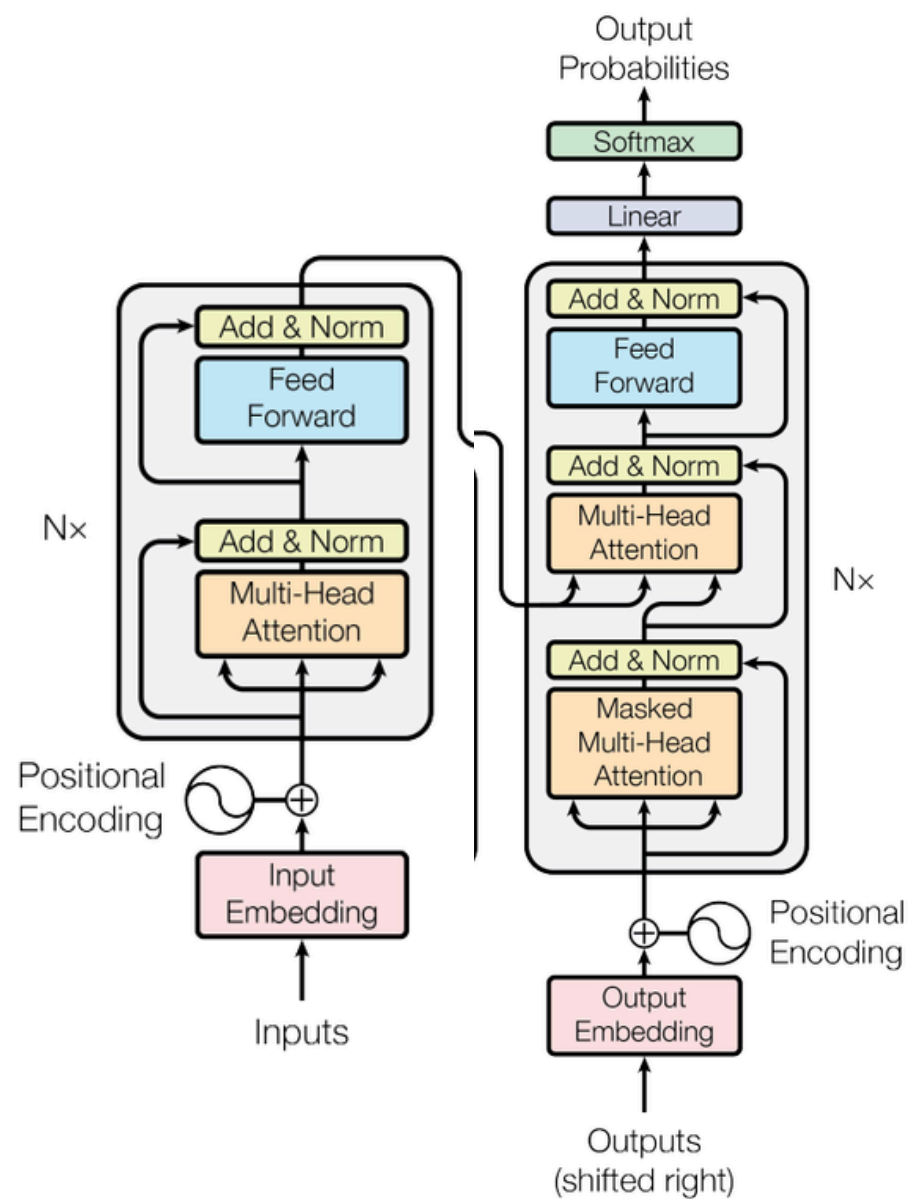
What task are we trying to do?

Understand or predict something
"I want to deeply understand what's here"



How to select which model to use?

What task are we trying to do?



How to select which model to use?

What task are we trying to do?

Predicting what comes next
"I want to predict what comes next"

Decoder only

GPT

LLaMA

